مسارات الـ EMV في EasyLink

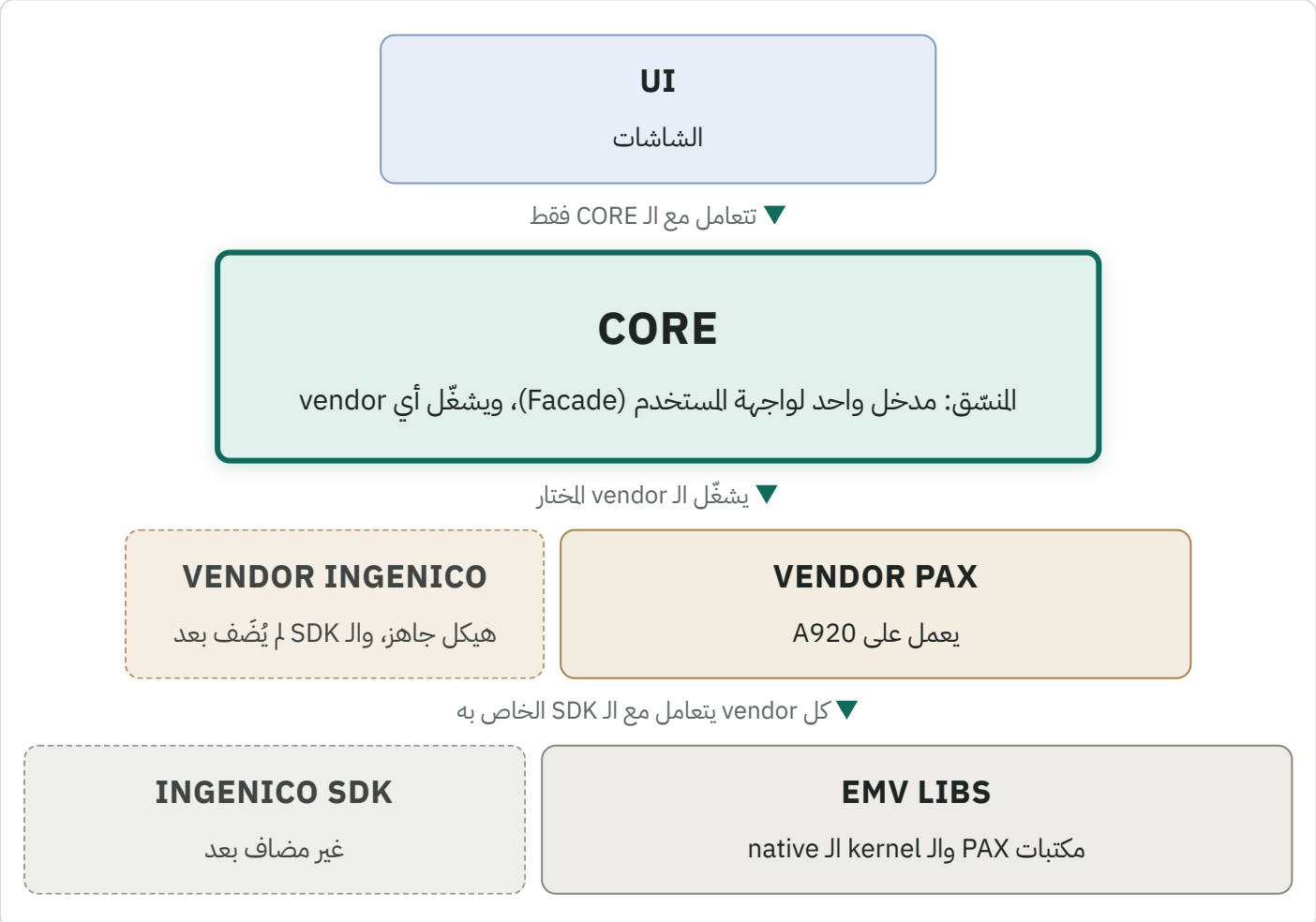
رحلة العملية من زر البيع حتى نهايتها: البطاقة، والـ kernel، والبنك، ثم التأكيد أو الإلغاء عند البنك. كل اسم في الصفحة موجود في الكود، وكل سهم استدعاء حقيقي. الشرح كله على فرع `transactionStepObserver`.

---

1. المكوّنات: من يملك ماذا
2. العملية من البداية إلى النهاية
3. خطوات الـ EMV: goToStep و onXxx
4. حين ينتظر الـ kernel شخصًا: askStep
5. الطريق إلى البنك
6. مسار الـ contact
7. النتيجة والنهاية الآمنة
8. مسار الـ contactless
9. الـ EMV params
10. مبادئ البرمجة في المشروع

1. المكوّنات: من يملك ماذا

لكل مسؤولية مالك واحد. الشاشة لا تعرف PAX، والـ EMV لا يعرف البنك ولا الـ threads، و `PosTerminal` هو الباب الوحيد الذي تدخل منه الشاشة.



`PosTerminal` هو الجهاز نفسه

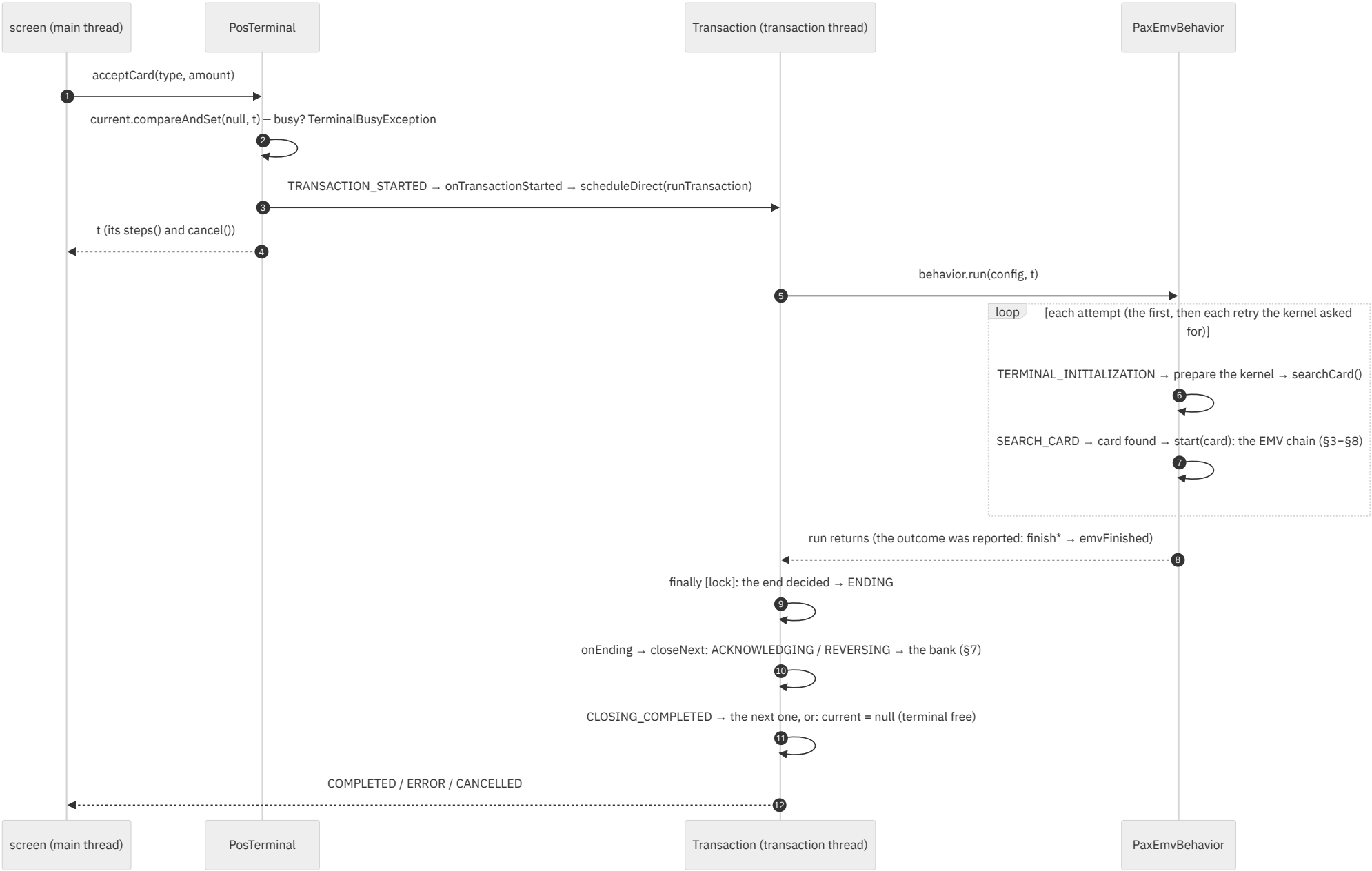
```
// PosTerminal = the POS machine
protected final AbstractEmvBehavior   behavior;       // the card: EMV (search + reader
protected final CommunicationBehavior communication;  // HTTP to the bank
protected final PrinterBehavior       printer;        // the receipt
private   final Scheduler             transactionScheduler; // one sale at a time, one t
```

الكلاس	مسؤول عن	ممنوع عليه
`PosTerminal`	عملية واحدة في كل مرة (`current` = مشغول)، والذهاب إلى البنك (`sendToBank`)، والطابعة	أي منطق EMV
`PosTerminal.Transaction`	حالة العملية كلها (`end`)، وخطواتها (`steps`)، والإلغاء، والطلبات المرسلة إلى البنك ونهايتها الآمنة	نشر أي شيء بعد انتهائها
`AbstractEmvBehavior`	الـ EMV المشترك: المحاولات، و `goToStep` ، و `askSte` p، و `askTheBank` / `bankAnswer` ، والـ 17 `onXx` x التي يكتبها الـ vendor	الـ threads، والانشغال، والبنك مباشرة
`PaxEmvBehavior`	الـ `onXxx` على PAX: استدعاء native لكل خطوة، وردود الـ kernel	أن يستدعي أحدٌ `onXxx` مباشرة
`EmvEngine`	شريط خطوات الـ EMV (`emvSteps`) والـ APDU trace	أي حالة للعملية
`SaleCommunicationBehavior`	الاتصال بالبنك: `connectToHost` (← token ← sale / refund exchange)، والتأكيد، والإلغاء؛ ويقرر هل وصل الطلب أم لا	أن يقرر التأكيد أو الإلغاء

2. العملية من البداية إلى النهاية

العملية تسير **بخطواتها** (TransactionSteps). كل خطوة تُنشر في `Transaction.steps` ، و
`TransactionStepObserver` هو المدخل الوحيد الذي يشغّل دالتها. ما ليس له دالة تقرير للشاشة فقط.

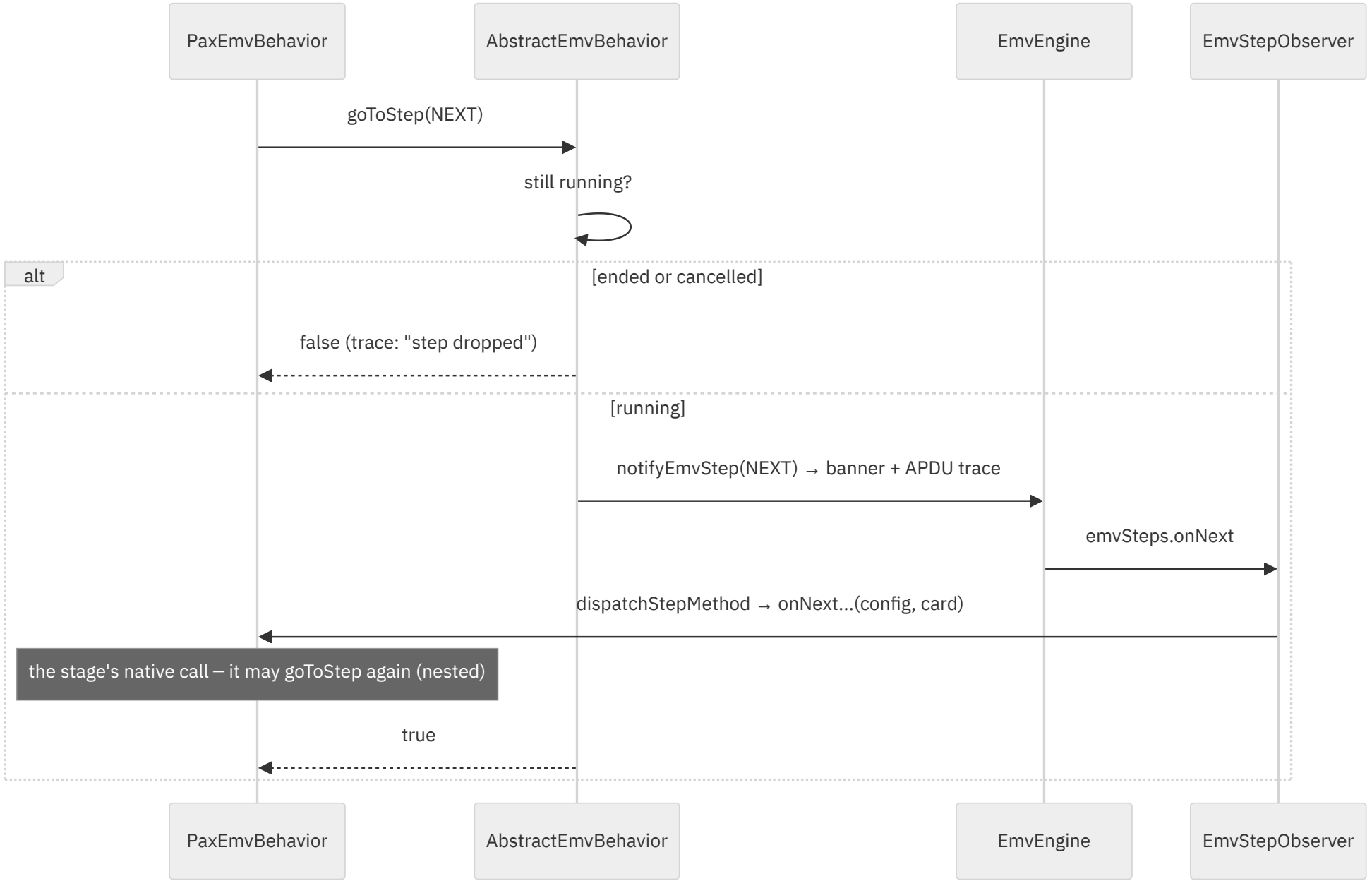
```
// TransactionStepObserver.onNext → dispatchTransactionStep(event)
switch (event.getStep()) {
    case TRANSACTION_STARTED: onTransactionStarted(event);   // → the transaction thread
    case CANCELLING:          onCancelling();                // stop the card search
    case ENDING:              onEnding();                    // the closings start
    case ACKNOWLEDGING:       onAcknowledging(event);        // tell the bank: done
    case REVERSING:           onReversing(event);            // ask the bank to reverse
    case CLOSING_COMPLETED:   onClosingCompleted(event);     // the next closing, or the
    default:                  // a report: ONLINE_*, APPROVED, DECLINED, COMPLETED / ERR
}
```


screen (main thread)
PosTerminal
Transaction (transaction thread)
PaxEmvBehavior
acceptCard(type, amount)
1
current.compareAndSet(null, t) — busy? TerminalBusyException
2
TRANSACTION_STARTED → onTransactionStarted → scheduleDirect(runTransaction)
3
t (its steps() and cancel())
4
behavior.run(config, t)
5
loop
[each attempt (the first, then each retry the kernel asked for)]
TERMINAL_INITIALIZATION → prepare the kernel → searchCard()
6
SEARCH_CARD → card found → start(card): the EMV chain (§3–§8)
7
run returns (the outcome was reported: finish* → emvFinished)
8
finally [lock]: the end decided → ENDING
9
onEnding → closeNext: ACKNOWLEDGING / REVERSING → the bank (§7)
10
CLOSING_COMPLETED → the next one, or: current = null (terminal free)
11
COMPLETED / ERROR / CANCELLED
12
screen (main thread)
PosTerminal
Transaction (transaction thread)
PaxEmvBehavior

- **الانشغال:** يُفحص على الـ main thread نفسه، فتُرفض عملية ثانية فورًا بـ `TerminalBusyException`، بلا طابور.
- **الإلغاء** (`Transaction.cancel()`): تُضبط النهاية `CANCELLED` وتُنشر `CANCELLING`، فتتوقف قراءة البطاقة، ويتوقف الـ EMV عند أول `goToStep`. لا يُقاطَع الـ thread أبدًا.
- **إعادة المحاولة** (fallback، tap again، See Phone): يطلبها الـ EMV بـ `requestRetry(...)` ثم يرجع الـ kernel، فيبدأ `run` محاولة جديدة من `TERMINAL_INITIALIZATION`، في العملية نفسها.
- **النتيجة ليست النهاية:** `APPROVED` / `DECLINED` تُنشر فور معرفتها، أما `COMPLETED` فبعد التأكيد / الإلغاء عند البنك وتحرير الجهاز.
- ترى الشاشة مثلًا: `ONLINE_REQUIRED → ONLINE_PROCESSING → ONLINE_COMPLETED → APPROVED → ENDING → ACKNOWLEDGING → CLOSING_COMPLETED → COMPLETED`.
- ⚠ دالة الخطوة تعمل والـ lock مأخوذ، فترى الشاشة `ACKNOWLEDGING` / `REVERSING` بعد انتهاء نداء البنك (BACKLOG 10.4). لهذا تبقى الخطوات التي قد تنتظر البنك أو شخصًا قبل تحديد النهاية (`ONLINE_PROCESSING`، و Single Tap مستقبلًا) تقارير لا دوال.

3. خطوات الـ EMV: كيف يشغّل `goToStep` الدالة `onXxx`

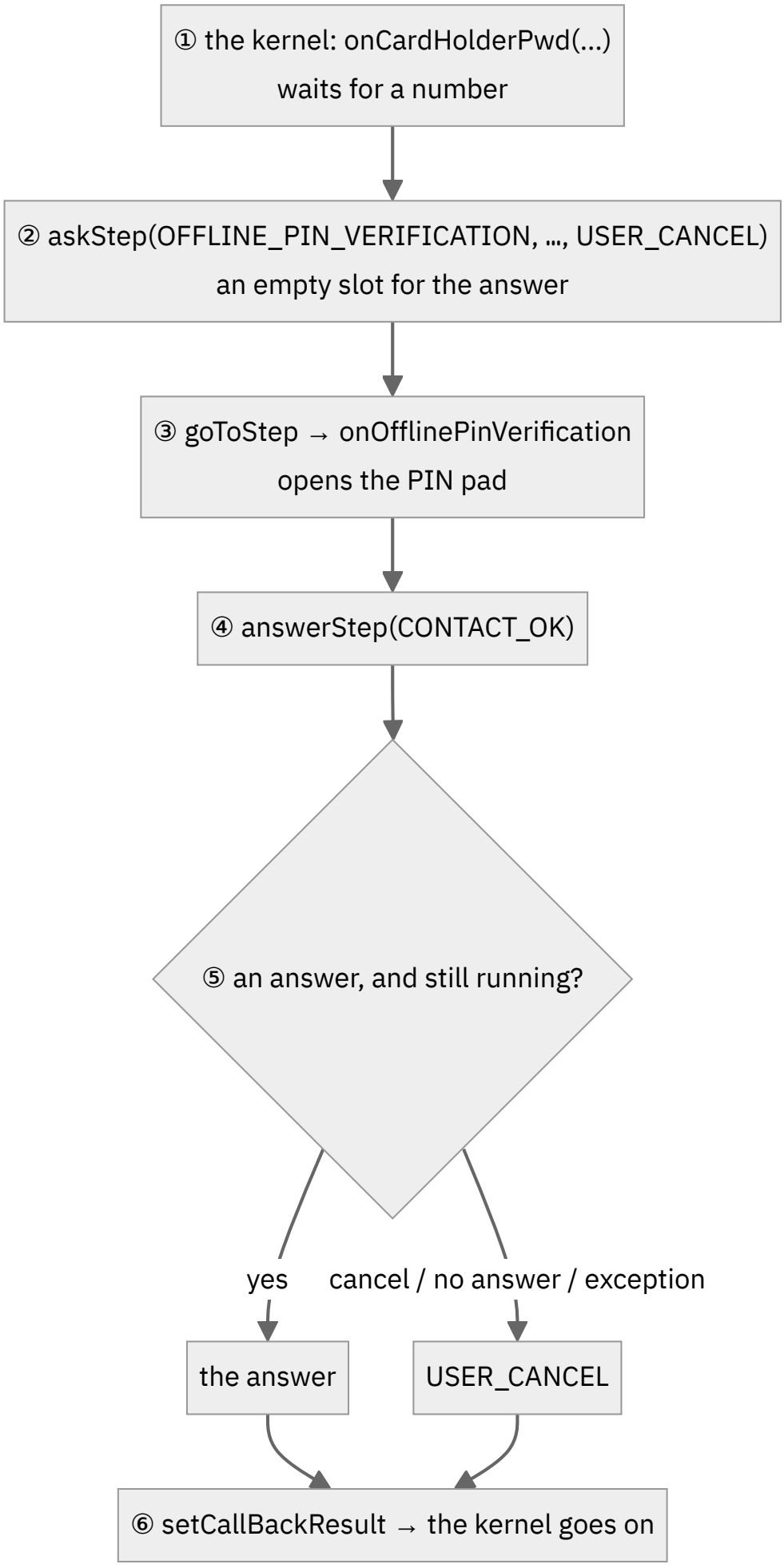
لا يستدعي أحد `onXxx` مباشرة. `goToStep(STEP)` تنشر الخطوة على `emvSteps`، فيشغّل `EmvStepObserver` دالتها على الـ thread نفسه، وتنتهي قبل أن تعود `goToStep`. إن كانت العملية قد انتهت أو أُلغيت تُسقط الخطوة وتعيد `goToStep` القيمة `false`.


PaxEmvBehavior
AbstractEmvBehavior
EmvEngine
EmvStepObserver
goToStep(NEXT)
still running?
alt
[ended or cancelled]
false (trace: "step dropped")
[running]
notifyEmvStep(NEXT) → banner + APDU trace
emvSteps.onNext
dispatchStepMethod → onNext...(config, card)
the stage's native call — it may goToStep again (nested)
true
PaxEmvBehavior
AbstractEmvBehavior
EmvEngine
EmvStepObserver

- **البداية:** `TERMINAL_INITIALIZATION` تجهّز الـ kernel ثم تنادي `searchCard()` بنفسها، فتبدأ `SEARCH_CARD`. إن فشل التجهيز تنتهي العملية بـ ERROR، ولا بحث.
- **يجب أن يبقى التشغيل متزامنًا** (بلا `observeOn` وبلا serialize): لو تأخرت الدالة إلى thread آخر لعادت `goToStep` قبل أن تعمل، فيرد `askStep` بـ "إلغاء" قبل أن تُفتح شاشة الـ PIN، وتتداخل استدعاءات native على kernel واحد.
- الشاشة وحدها تستخدم `observeOn(main)` على `emvSteps`، لأنها تعرض الشريط فقط ولا ينتظرها أحد.

4. حين ينتظر الـ kernel شخصًا: `askStep`

أثناء استدعاء native (مثل `EMVStartTrans`) يتوقف الـ kernel ويسأل سؤالًا عبر callback: اختر التطبيق، أدخل الـ PIN. يجب أن يعود له ردٌّ دائمًا (`setCallBackResult`)، وإلا يبقى منتظرًا. الـ callback لا يفتح شاشة بنفسه؛ يسأل الخطوة المسؤولة عبر `askStep`.

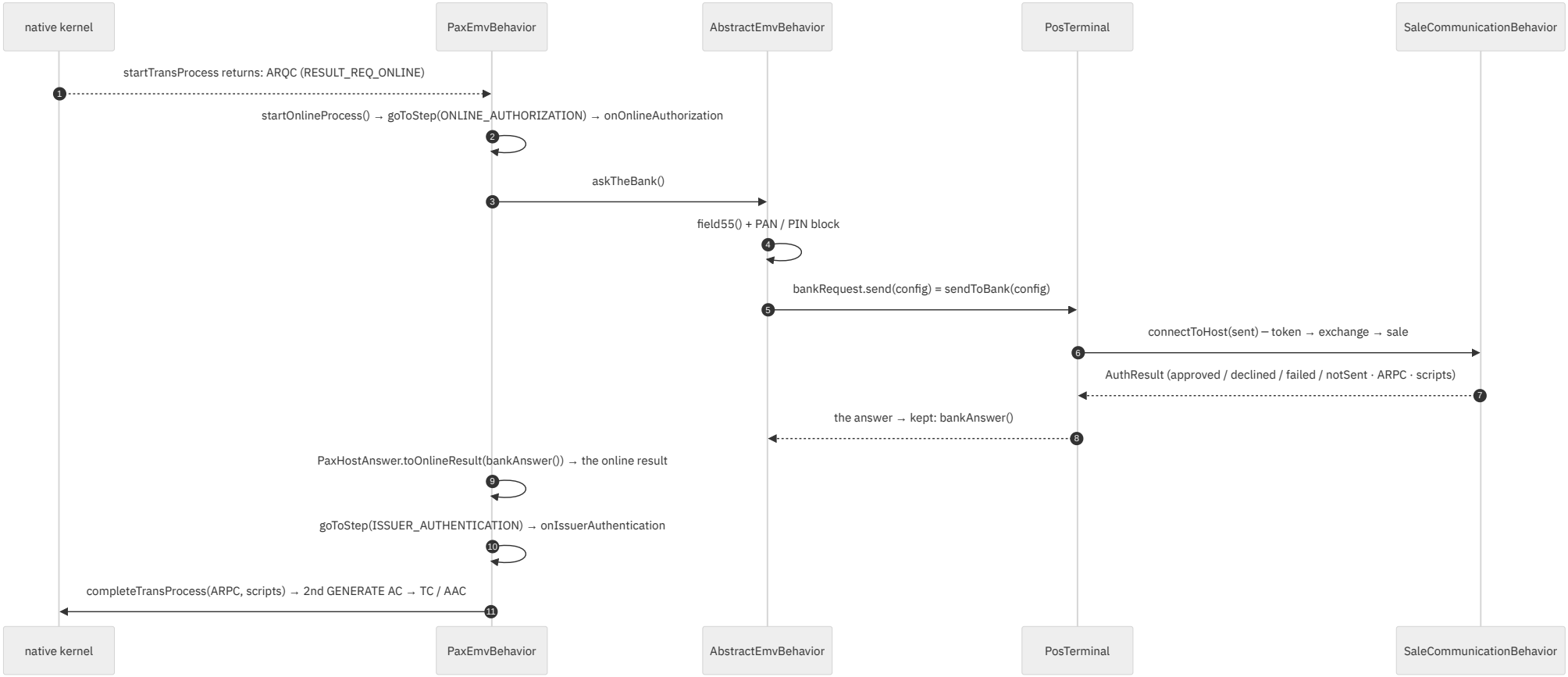

① the kernel: onCardHolderPwd(...)
waits for a number
② askStep(OFFLINE_PIN_VERIFICATION, …, USER_CANCEL)
an empty slot for the answer
③ goToStep → onOfflinePinVerification
opens the PIN pad
④ answerStep(CONTACT_OK)
⑤ an answer, and still running?
yes
cancel / no answer / exception
the answer
USER_CANCEL
⑥ setCallBackResult → the kernel goes on

- قيمتها في الحالات غير الطبيعية: إلغاء، أو لا رد، أو استثناء، فيصل إلى الـ kernel "إلغاء" فورًا ولا يبقى منتظرًا، ولا يوافق أبدًا دون رد (fail closed).
- الـ callbacks التي تسأل: `onWaitAppSelect` (اختيار التطبيق)، و `onCardHolderPwd` (الـ PIN). أما callbacks البيانات (`emvSetParam` ، `emvInputAmount`) فترد داخل `ContactProcess` دون أي خطوة.

5. الطريق إلى البنك

حين يرد الكارت بـ ARQC في الـ 1st GENERATE AC، **ينتهي الاستدعاء الـ native ويرجع** بالنتيجة `RESULT_REQ_ONLINE`. بعدها كودنا هو الذي يذهب إلى البنك، ثم يعود إلى الـ kernel بالـ 2nd GENERATE AC في خطوة أخرى. ثلاث طبقات، لكل منها اسمها:

الطبقة	الاسم	ماذا تفعل
الـ EMV	`askTheBank()` (خطوة `ONLINE_AUTHORIZATIO N`)	يجهّز field 55 وبيانات البطاقة، ويسأل البنك، ويحفظ الرد: `bankAnswer()`
الجهاز	`PosTerminal.sendToBank(config)`	يسمّي الطلب (`asyncRequestId`) ويسجّله للنهاية الآمنة، وينشر `ONLINE_*`
البنك	`CommunicationBehavior.connectToHost`	HTTP عبر token ← exchange ← sale / refund


native kernel
PaxEmvBehavior
AbstractEmvBehavior
PosTerminal
SaleCommunicationBehavior
startTransProcess returns: ARQC (RESULT_REQ_ONLINE)
1
startOnlineProcess() → goToStep(ONLINE_AUTHORIZATION) → onOnlineAuthorization
2
askTheBank()
3
field55() + PAN / PIN block
4
bankRequest.send(config) = sendToBank(config)
5
connectToHost(sent) — token → exchange → sale
6
AuthResult (approved / declined / failed / notSent · ARPC · scripts)
7
the answer → kept: bankAnswer()
8
PaxHostAnswer.toOnlineResult(bankAnswer()) → the online result
9
goToStep(ISSUER_AUTHENTICATION) → onIssuerAuthentication
10
completeTransProcess(ARPC, scripts) → 2nd GENERATE AC → TC / AAC
11
native kernel
PaxEmvBehavior
AbstractEmvBehavior
PosTerminal
SaleCommunicationBehavior

كيف يعرف الـ EMV طريق البنك: `attach`

```
// BankRequest.java
public interface BankRequest { AuthResult send(TransactionConfig config); }

// AbstractEmvBehavior
private BankRequest bankRequest;
public final void attach(BankRequest bankRequest) { this.bankRequest = bankRequest; }
protected final void askTheBank() { bankAnswer = callTheBank(); }   // … bankRequest.sen
protected final AuthResult bankAnswer() { return bankAnswer; }      // read it, never as

// PosTerminal, in its constructor, once
behavior.attach(this::sendToBank);
```

- **لماذا `attach` وليس الـ constructor؟** دائرة: `PosTerminal` يحتاج الـ behavior في الـ constructor الخاص به، والـ behavior يحتاج `PosTerminal::sendToBank` الذي لا يوجد إلا بعده. فيُنشأ الـ behavior أولًا، ثم يوصّله `PosTerminal` بالبنك مرة واحدة (setter injection). Dagger لا يحل هذه الدائرة؛ حلّه هو `attach` نفسها خلف annotations.
- **الـ EMV لا يعرف `PosTerminal`:** معه `BankRequest` فقط (Dependency Inversion). الـ field `private` و `attach` `final`، فلا يغيّر أحد الطريق ولا يتجاوز الفحوص.
- **نسختان من الرد، لكلٍّ مالك:** `bankAnswer()` للـ EMV (ليكمل مع البطاقة، وتُمسح مع كل محاولة)، و `OnlineRequest.answer` لـ `PosTerminal` (ليقرر التأكيد أو الإلغاء). لا يقرأ أحدهما نسخة الآخر.
- **`startOnlineProcess()` ليست callback من الـ kernel:** هي method في واجهات PAX (`IContactCallback` / `IContactlessCallback`) يناديها كودنا بعد أن يرجع الـ kernel. تبقى باسمها لأنها من الـ SDK.
- الـ swipe والإدخال اليدوي لا kernel لهما: `onReadApplicationData` يذهب مباشرة إلى `ONLINE_AUTHORIZATION`، ثم `TRANSACTION_COMPLETION` يقرر من `bankAnswer()`.

6. مسار الـ contact (بترتيب الـ kernel نفسه)

كل مرحلة = استدعاء native واحد = `EmvStep` واحدة. كل `runContact*` تستدعي الـ kernel: إن نجحت تنتقل بـ `advanceContactStage(NEXT)`، وإلا تنهي المسار بـ `finishContactStage()` (تحرير الـ listener، وإغلاق القارئات، وحساب النتيجة).

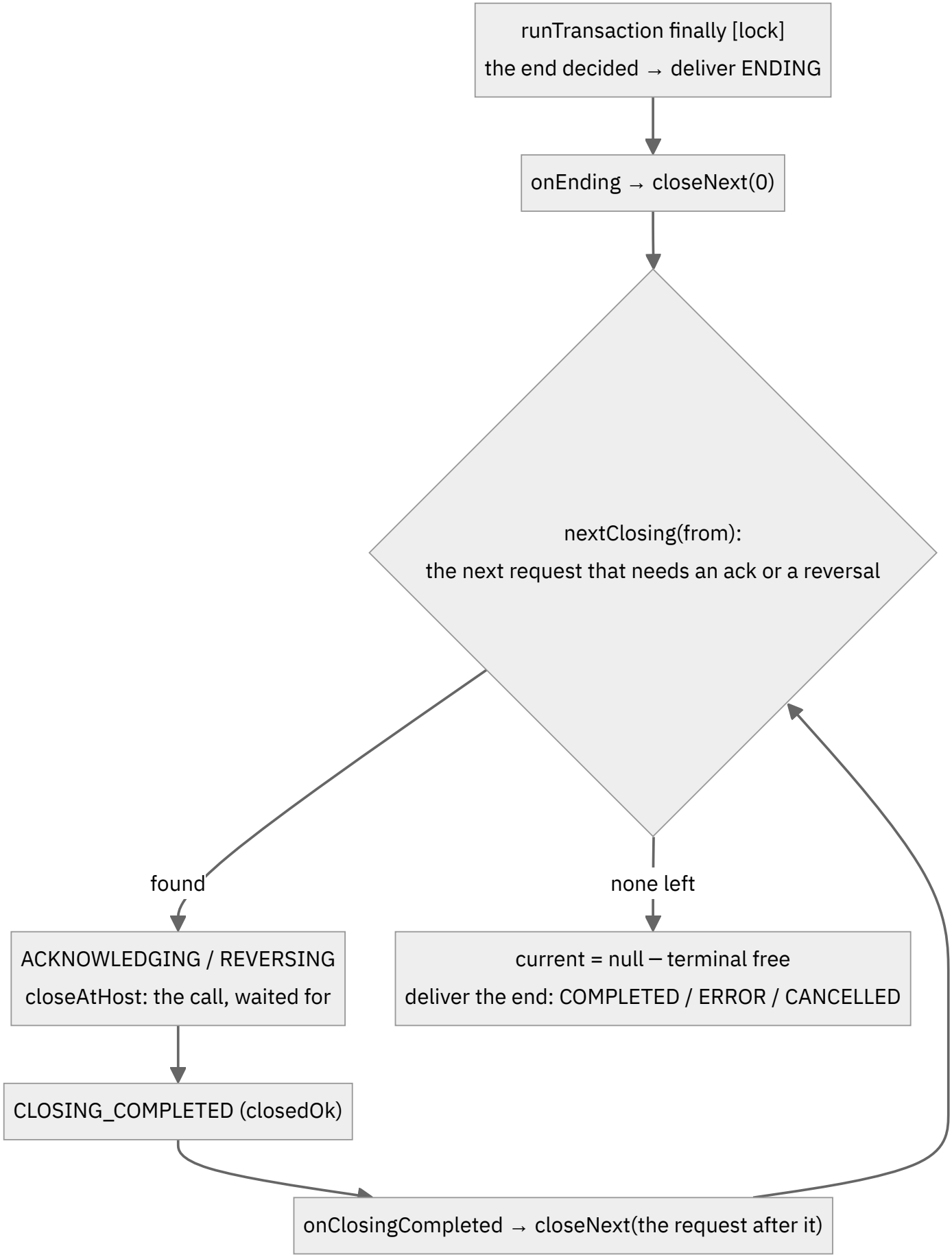
#	EmvStep	الدالة في PAX	native	يسأل شخصًا؟
0	`TERMINAL_INITIALIZATION`	`preTransProcess` → `prepareKernel`	`EMVCoreInit`، `EMVAddApp`	—
1	`SEARCH_CARD`	`PaxCardSearch` → `onSearchCard`	polling	—
2	`APPLICATION_SELECTION`	`runContactAppSelect`	`EMVAppSelect`	اختيار التطبيق
5	`READ_APPLICATION_DATA`	`runContactReadAppData`	`EMVReadAppData` (+ GPO records)	—
6	`SET_TRANSACTION_DATA`	`runContactSetTransactionData`	`CARD_READ` · 9C	—
7	`OFFLINE_DATA_AUTHENTICATION`	`runContactCardAuth`	`EMVAddCAPK`، `EMVCardAuth`	—
8	`PROCESS_RESTRICTIONS`	`runContactStartTransaction`	`EMVStartTrans` = PR + CVM + TRM + TAA + 1st GAC	الـ PIN
13	`ONLINE_AUTHORIZATION`	`startOnlineProcess` → `onOnlineAuthorization`	— (البنك)	—
14	`ISSUER_AUTHENTICATION`	`runIssuerAuthentication`	`EMVCompleteTrans` = ARPC + scripts + 2nd GAC	—
16	`TRANSACTION_COMPLETION`	`finishWithOutcome` ← `reportOutcome`	—	—

- **متى يذهب الطلب إلى البنك:** حين يرد الكارت بـ ARQC. الاسترداد يطلب ARQC كالبيع. الاستثناء: TC على جهاز online فقط، فيُرسل دون ARQC ويكون رد البنك هو النتيجة. AAC = رفض. (`RESULT_OFFLINE_APPROVED`)
- **الـ 2nd GENERATE AC دائمًا** بعد البنك (AET-146)، حتى لو رفض البنك أو لم يرد.
- في الـ contact، الـ scripts تعمل داخل `EMVCompleteTrans` نفسه، فـ `SCRIPT_PROCESSING` هنا مجرد عنوان، وليست مرحلة منفصلة.
- ⚠ `TERMINAL_RISK_MANAGEMENT` و `TERMINAL_ACTION_ANALYSIS` لا تُنشران: لا يوجد لهما استدعاء native منفصل (داخل `EMVStartTrans`).

7. النتيجة والنهاية الآمنة

الـ EMV يبلّغ بالنتيجة (`finishApproved` / `finishDeclined` / `finishError`) → `Transaction.emvFinished`)، والـ `Transaction` هي التي تنهي العملية. وقبل أن يصبح الجهاز متاحًا، يُغلق كل طلب أُرسل إلى البنك: **تأكيد** (acknowledgement) أو **إلغاء** (reversal).

البنك	البطاقة في الـ 2nd GAC	النتيجة	عند البنك بعدها
وافق	TC	APPROVED → COMPLETED	تأكيد
وافق	AAC	DECLINED → COMPLETED	إلغاء
رفض	—	DECLINED → COMPLETED	لا شيء
لا رد / انتهت المهلة (`failed`)	AAC	ERROR	إلغاء (ربما وافق)
لم يُرسل (`notSent`)	AAC	ERROR	لا شيء (لم يصل)


runTransaction finally [lock]
the end decided → deliver ENDING
onEnding → closeNext(0)
nextClosing(from):
the next request that needs an ack or a reversal
found
none left
ACKNOWLEDGING / REVERSING
closeAtHost: the call, waited for
current = null — terminal free
deliver the end: COMPLETED / ERROR / CANCELLED
CLOSING_COMPLETED (closedOk)
onClosingCompleted → closeNext(the request after it)

من يقرر `notSent` ؟ الـ network وحده

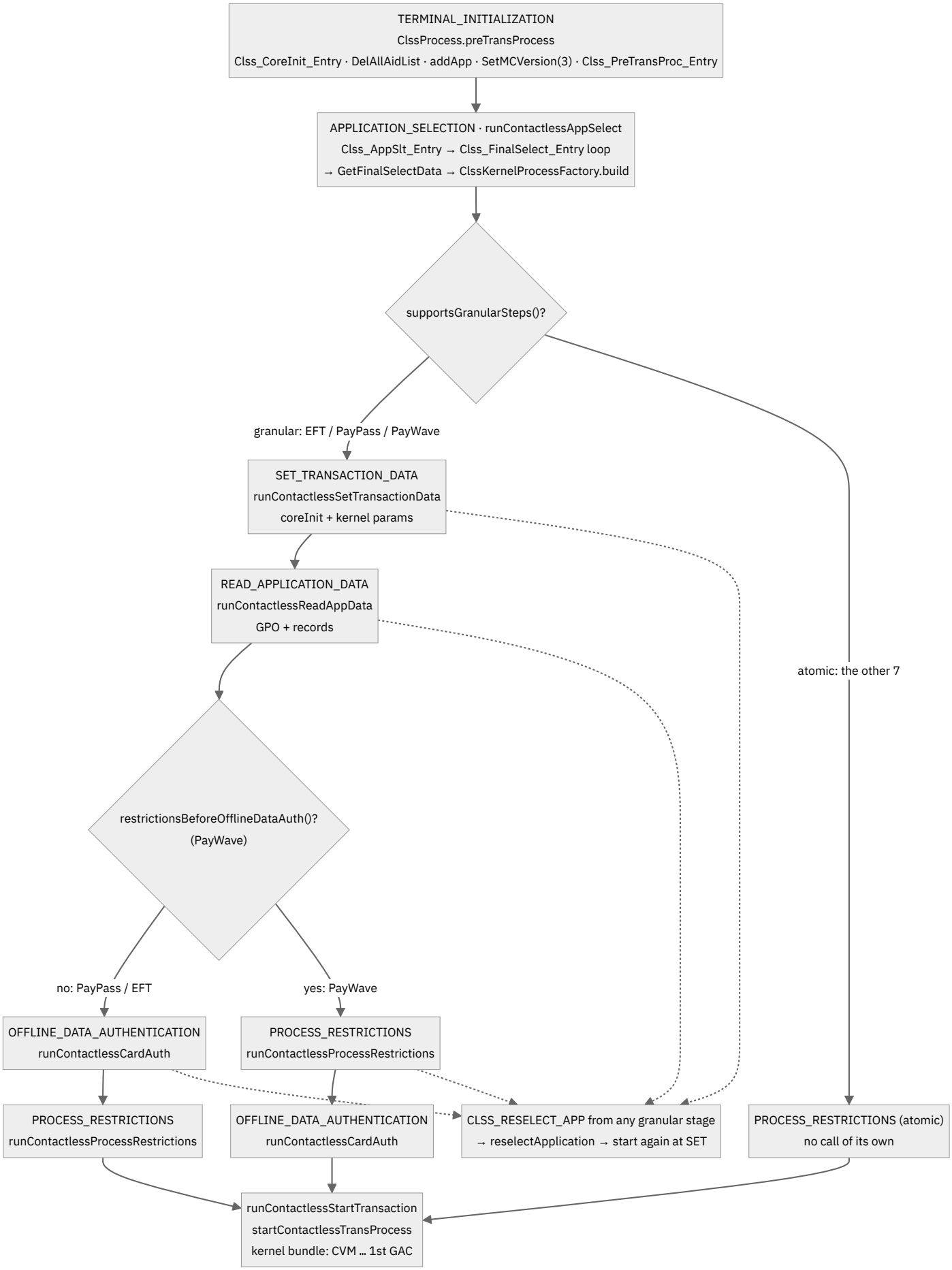
أين فشل	النتيجة	إلغاء؟
قبل الإرسال: لا مرجع للاسترداد، لا EMV result / مفتاح / token، فشل التشفير	`notSent`	لا
`Token refresh: …` أو `Exchange: …`	`notSent`	لا
`Sale:` لا اتصال (`ConnectException` / `Unknow` `nHost`) أو HTTP 401 / 403	`notSent`	لا
`Sale:` انتهاء المهلة، انقطاع، 4xx، 5xx، envelope آخر، تالف	`failed`	نعم

المهلة	القيمة	ملاحظة
كل نداء دفع (OkHttp `callTimeout`)	25 ث	token + exchange + sale ≤ 75 ث؛ للدفع فقط، لا لتنزيل الـ params
`PosTerminal.sendToBank`	90 ث	شبكة أمان فوق الـ 75
`closeAtHost` (تأكيد / إلغاء)	60 ث	شبكة أمان؛ الحد الفعلي 25 ث للنداء

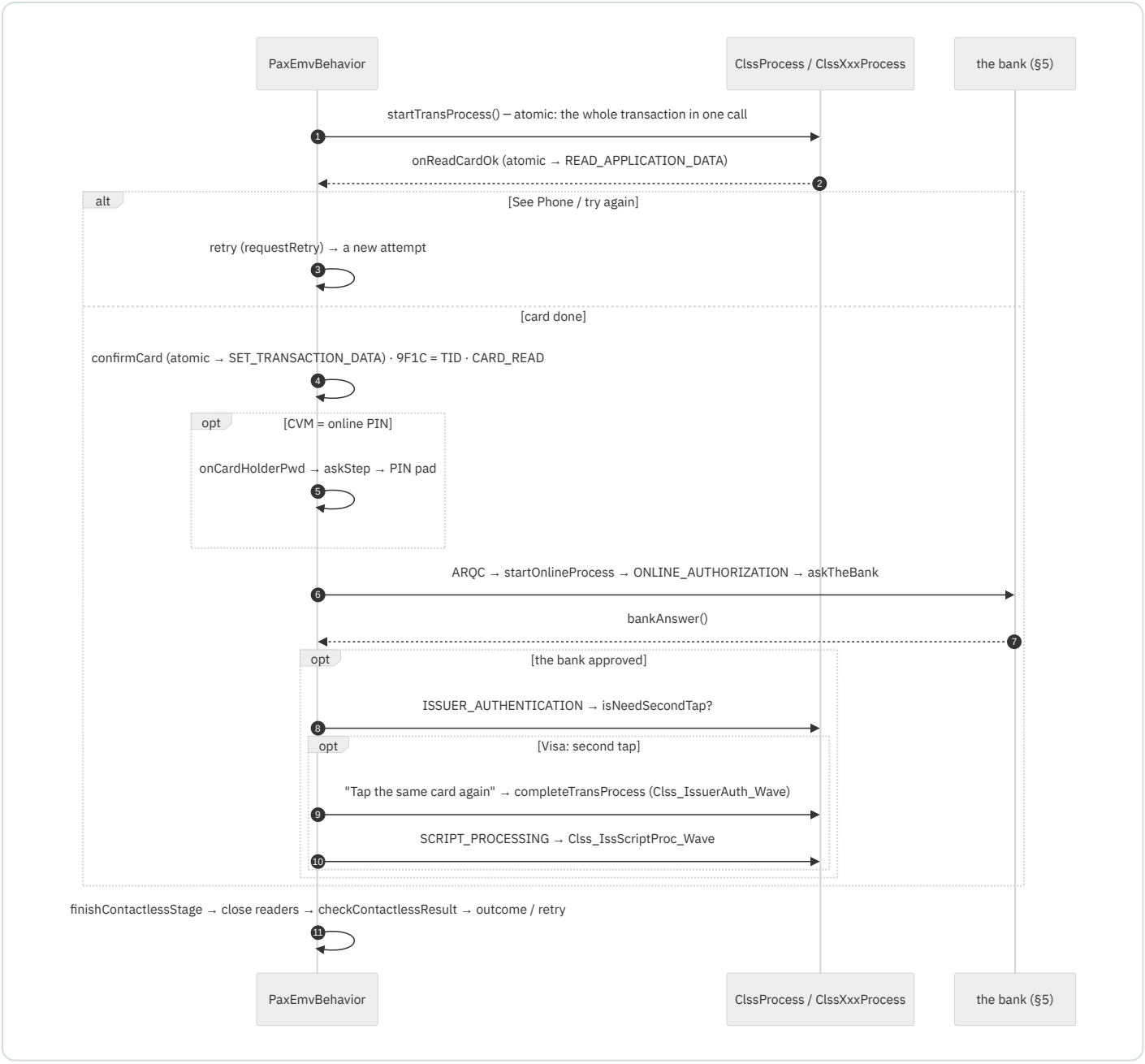
- نداءات الدفع تستخدم الـ adapter المتزامن (`createSynchronous()`)، وإلا يبتلع Retrofit الخطأ حين يقطع `callTimeout` النداء، فيبقى الجهاز مشغولًا للأبد (أُصلح في `1a97dee`).
- ⚠ التأكيد / الإلغاء يُرسل مرة واحدة: إن فشل يُسجَّل في الـ trace فقط، بلا سجل محفوظ لإعادة الإرسال (BACKLOG 0.15).

8. مسار الـ contactless

طريقة التشغيل نفسها و `askStep` نفسها، لكن الـ kernel يُختار بعد اختيار التطبيق (`ClssKernelProcessFactory`). نوعان: **granular** (PayPass و PayWave و EFT) ولكل مرحلة استدعاء native مستقل، و**atomic** (Amex و JCB و Mir و PBOC و Pure و RuPay و DPAS) وفيها استدعاء واحد يعمل كل شيء.


TERMINAL_INITIALIZATION
ClssProcess.preTransProcess
Clss_CoreInit_Entry · DelAllAidList · addApp · SetMCVersion(3) · Clss_PreTransProc_Entry
APPLICATION_SELECTION · runContactlessAppSelect
Clss_AppSlt_Entry → Clss_FinalSelect_Entry loop
→ GetFinalSelectData → ClssKernelProcessFactory.build
supportsGranularSteps()?
granular: EFT / PayPass / PayWave
atomic: the other 7
SET_TRANSACTION_DATA
runContactlessSetTransactionData
coreInit + kernel params
READ_APPLICATION_DATA
runContactlessReadAppData
GPO + records
restrictionsBeforeOfflineDataAuth()?
(PayWave)
no: PayPass / EFT
yes: PayWave
OFFLINE_DATA_AUTHENTICATION
runContactlessCardAuth
PROCESS_RESTRICTIONS
runContactlessProcessRestrictions
PROCESS_RESTRICTIONS
runContactlessProcessRestrictions
OFFLINE_DATA_AUTHENTICATION
runContactlessCardAuth
CLSS_RESELECT_APP from any granular stage
→ reselectApplication → start again at SET
PROCESS_RESTRICTIONS (atomic)
no call of its own
runContactlessStartTransaction
startContactlessTransProcess
kernel bundle: CVM … 1st GAC

بعد المراحل: `startContactlessTransProcess`

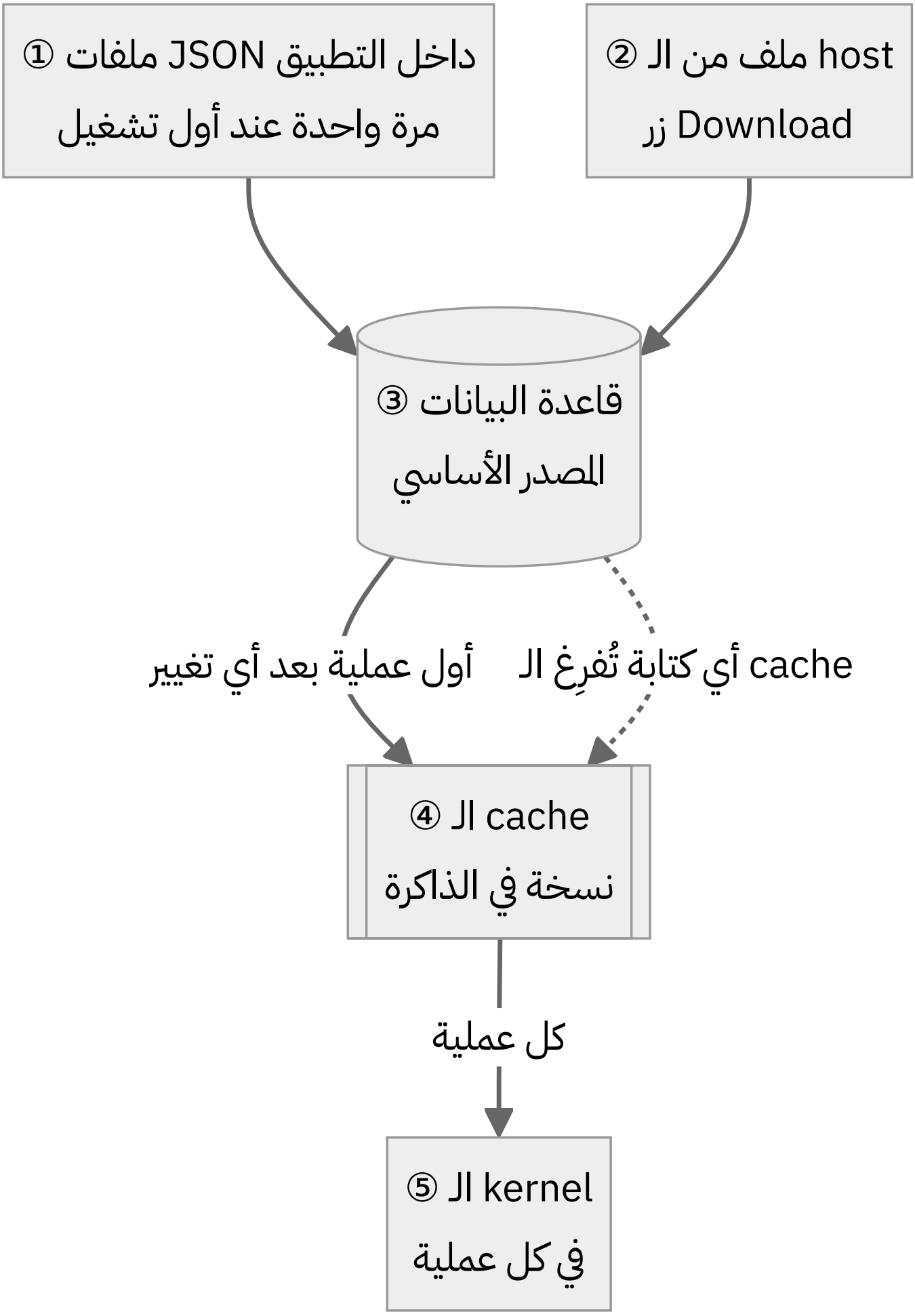


- **الترتيب يتبع الـ kernel، لا الأرقام:** في الـ atomic تُعلَن `READ_APPLICATION_DATA` و `SET_TRANSACTION_DATA` من الـ callbacks في اللحظة التي يقول فيها الـ kernel إنها حدثت، ولا تُعاد القراءة (الـ hooks لا تعمل شيئًا للـ atomic).
- إذا لم يوافق البنك، فلا issuer authentication ولا 2nd GENERATE AC: البطاقة غادرت الحقل.
- ⚠ رسالة الـ tap الثاني لا تنتظر البطاقة قبل `Clss_FinalSelect_Entry` (BACKLOG 1.4).

EmvStep	PayPass (`ClssPayPassProcess`)	PayWave (`ClssPayWaveProcess`)	EFT (`ClssEFTProcess`)
`SET_TRANSACTION_DATA`	`Clss_CoreInit_MC` ، `SetParam_M` FinalSelectData + `setPayPass` ،`C Param` (DF8117/18/19/1F، limits)	`CoreInit_Wave` ، ReaderParam `SetVisaAi` + (`aucTmCap` = 9F33) `SetTransData_Wav` ،`dParam_Wave e`	`CoreInit_EFT` + + ReaderParam `SetTransData_EFT` + AidParam
`READ_APPLICATION_DATA`	`Clss_InitiateApp_MC` + `Clss_R` `eadData_MC`	`Clss_Proctrans_Wave` (+ GPO (records + AC type	`Clss_InitApp_EFT` + `Clss_Read` `AppData_EFT`
الترتيب بعدها	ODA ثم PR	**PR ثم ODA**	ODA ثم PR
`OFFLINE_DATA_AUTHENTICATION`	تحميل CAPK و revocation فقط (الـ ODA داخل `TransProc`)	`Clss_CardAuth_Wave` (عند TC وليس استردادًا، أو حسب TTQ)	`PreTAAProc` + CAPK + `Clss_Car` `dAuth_EFT`
`PROCESS_RESTRICTIONS`	لا يوجد استدعاء	`Clss_ProcRestric_Wave`	`Clss_ProcRestrict_EFT`
الـ bundle	`Clss_TransProc_MC_MChip` / `_M` = `ag` ODA+PR+CVM+TRM+TAA+GAC، ثم see phone	`genTransResult` : `Clss_GetCvmT` AC type + `ype_Wave`	`Clss_CVMProc_EFT` + `Clss_Post` `TAAProc_EFT`
2nd tap / completion	`completeTransProcess`	FinalSelect + `Clss_IssuerAuth_W` `ave`	`Clss_CompleteOnlineTrans_EFT` ⚠ لا يُستدعى
`SCRIPT_PROCESSING`	default	`Clss_IssScriptProc_Wave`	default

9. الـ EMV params: مصدرها، ومكان حفظها، ومتى تصل إلى الـ kernel

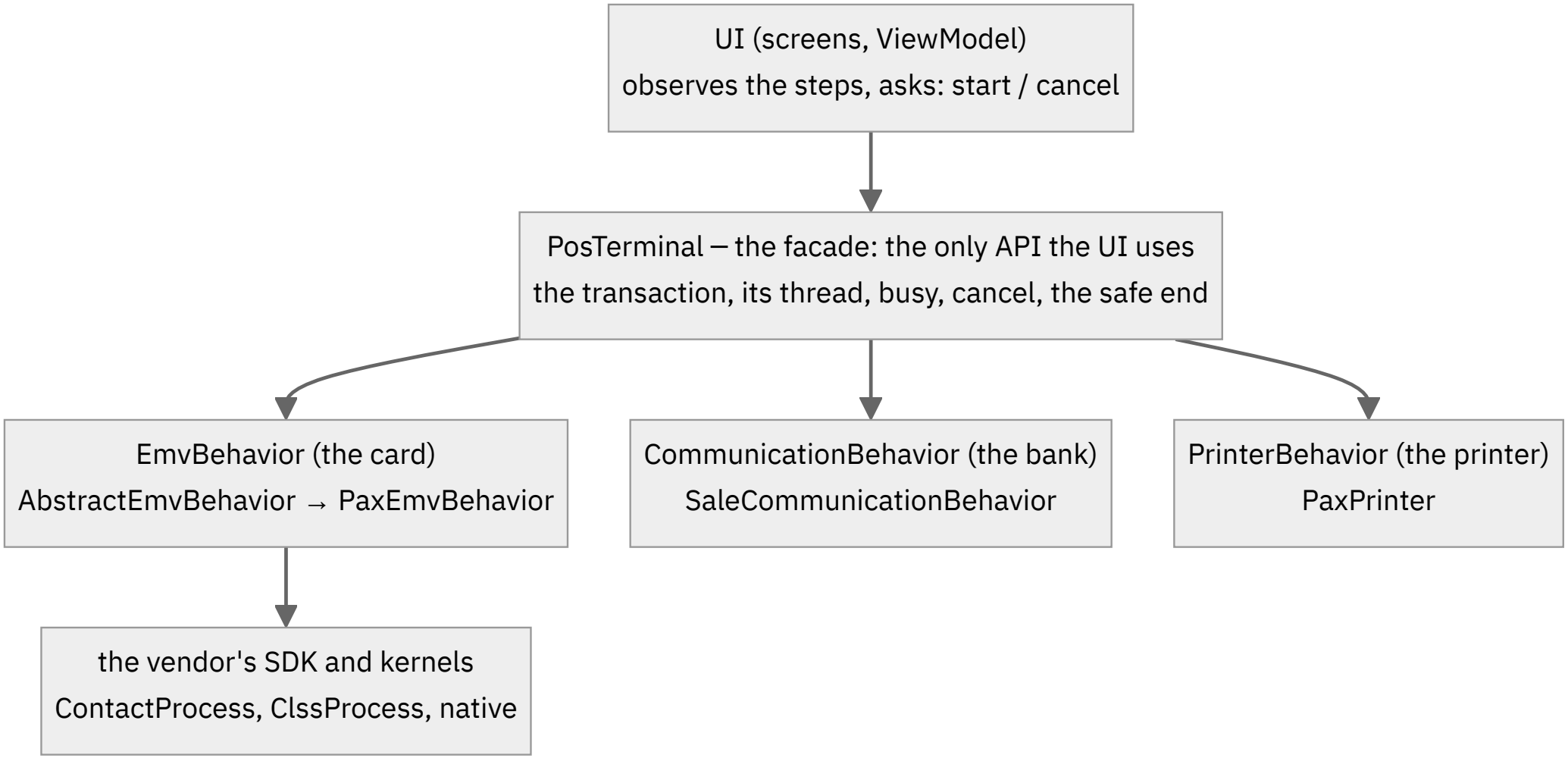
الـ params هي ما يحتاجه الـ kernel ليقرر: البطاقات المقبولة (AIDs)، ومفاتيح التحقق من البطاقة (CAPKs)، والحدود المالية (مثل المبلغ الذي يُطلب بعده PIN).


① ملفات JSON داخل التطبيق
مرة واحدة عند أول تشغيل
② ملف من الـ host
زر Download
③ قاعدة البيانات
المصدر الأساسي
أول عملية بعد أي تغيير
أي كتابة تُفرِغ الـ cache
④ الـ cache
نسخة في الذاكرة
كل عملية
⑤ الـ kernel
في كل عملية

- **قاعدة البيانات هي المصدر:** تُملأ من JSON مرة واحدة عند أول تشغيل، ثم من ملف الـ host عند Download (`EmvParamUpdater.applyFromZip`).
- **الـ cache نسخة في الذاكرة:** أي كتابة في قاعدة البيانات تفرغه، وأول عملية بعدها تملؤه (`getCachedEmvParam` → `loadFromDatabase`).
- **الـ kernel لا يحتفظ بشيء:** في كل عملية تُحمَّل إليه الـ params من الـ cache (`preTransProcess`).
- الـ contact: ما لا يأتي في ملف الـ host يبقى. الـ contactless: ملف الـ host هو المرجع، وأي kernel غير موجود فيه يُحذف.
- بيانات الجهاز (MCC، MID، TID، اسم التاجر) لا تأتي من ملف الـ host.
- شاشة EMV Params: **DB** = المحفوظ، **Cached** = ما سيستلمه الـ kernel الآن (فارغ = لم تتم عملية منذ آخر تغيير).
- ⚠ لا فحص لصلاحية الـ CAPKs (76 من 88 في JSON منتهية).

10. مبادئ البرمجة في المشروع (ببساطة)

كل مبدأ في سطر: معناه ببساطة، وأين يظهر في الكود. لا شيء هنا للزينة؛ كل واحد يحل مشكلة حقيقية في الدفع.


UI (screens, ViewModel)
observes the steps, asks: start / cancel
PosTerminal — the facade: the only API the UI uses
the transaction, its thread, busy, cancel, the safe end
EmvBehavior (the card)
AbstractEmvBehavior → PaxEmvBehavior
CommunicationBehavior (the bank)
SaleCommunicationBehavior
PrinterBehavior (the printer)
PaxPrinter
the vendor's SDK and kernels
ContactProcess, ClssProcess, native

المبدأ / النمط	ببساطة	أين في الكود
Layers + Facade	طبقات، كل طبقة تكلم التي تحتها فقط. والـ UI يرى بابًا واحدًا يخفي ما خلفه	`PosTerminal` هو الباب الوحيد للـ UI (`startTran` / `steps()` / `acceptCard` / `saction()` / `cancel`). ليس طبقة تمرير فارغة: يملك العملية فعلًا (القاعدة 5 تمنع الطبقات التي تمرّر فقط)
Strategy / Polymorphism	واجهة واحدة وأكثر من تنفيذ، يُختار واحد منها	`EmvBehavior` (PAX / Ingenico / Fake)، `Commu`nicationBehavior، `PrinterBehavior`
Open / Closed	نضيف بدل أن نعدّل	`TerminalFactory` جديد بـ flavor = vendor جديد وكلاساته، والـ core لا يتغيّر
Template Method	الكلاس الأب يحدد الترتيب، والابن يملأ الخطوات	`AbstractEmvBehavior` : `run` و `goToStep` والـ dispatch، و 17 `onXxx` يملؤها الـ vendor؛ `askThe`; `final` `Bank()`
Observer	من يهمه الأمر يشترك، ويصله كل جديد	`Transaction.steps()` (`ReplaySubject`) والشاشة مشتركة؛ `EmvEngine.emvSteps` و `EmvSt`epObserver؛ `TransactionStepObserver`
Callback	تعطي جهة "رقمًا تتصل بك عليه"	`BankRequest` (§5)، `CardSearchListener`، `I`ContactCallback و `askStep` (§4)
Adapter	محوّل بين شكلين لا يعرف أحدهما الآخر	`PaxHostAnswer` (`AuthResult` → شكل PAX)؛ `ContactProcess.EmvCallBackListener` (native → listener)؛ `PaxEmvBehavior` بصفته `I`ContactCallback
Factory / Builder	مكان واحد يقرر كيف يُنشأ الشيء	`TerminalFactory` (لكل flavor)، `ClssKernelPr`ocessFactory (لكل kernel scheme)، `AuthResu`lt.approved / declined / failed / notSen`t`، `TransactionStepEvent.builder(...)`
Dependency Injection (يدوي)	الكلاس يأخذ ما يحتاجه من الخارج ولا يُنشئه	`PaxTerminal` → `super(behavior, communic`ation, printer)؛ و `attach(this::sendToBa`nk) للدائرة الوحيدة
Dependency Inversion (الـ D في SOLID)	الطبقة العليا تعتمد على واجهة، لا على كلاس محدد	`AbstractEmvBehavior` يعرف `BankRequest`avior، لا `PosTerminal` ؛ الـ core يعرف `CommunicationBeh`Retrofit، لا
Single Responsibility	كل كلاس له شغلة واحدة	`PaxCardSearch` (البحث)، `PaxPinEntry` (الـ PIN)، `PaxHostAnswer` ، `PaxReaders` ، `PaxCar`dData، `PaxSetup` ، `PaxKernelSession`
Single source of truth	كل حالة لها مالك واحد ونسخة واحدة	الجهاز مشغول = `PosTerminal.current` ؛ حالة العملية كلها = `Transaction.end` ؛ خطواتها = `Tran`saction.steps
Immutability / Value Object	الشيء لا يتغيّر بعد إنشائه؛ التعديل يعطي نسخة جديدة	`AuthResult` ، `TransactionStepEvent` ، `Tran`sactionConfig.withXxx(...) يرجع config جديدًا
Fail closed	عند الشك نرفض، لا نوافق	لا رد = online failed؛ لا PIN = `USER_CANCEL` ؛ الـ callback يرد دائمًا (`setCallBackResult`)

ملحوظة: الفرق مع `paxEmvBehaviorSplit`

الفرع الأقدم `paxEmvBehaviorSplit` (ومعه `-tests`) يعمل بالطريقة نفسها مع أسماء وتفاصيل أقدم: `authorizeOnline` / `Authorizer` / `PosTerminal.authorize` / `CommunicationBehavior.authorize` ؛ و `START_ONLINE_PROCESS` ؛ و `tx.lastAuth` في كل vendor؛ و `CLOSED` بدل `CLOSING_COMPLETED` ؛ ولا `ENDING` (النهاية عبر `onEnd` وحلقة)؛ والـ switch تستدعيه أكثر من جهة؛ والـ network بسلسلة Rx (`flatMap`) بلا `callTimeout` . القائمة الكاملة في `doc/REFERENCE.md` .